

# Research Proposal Assessment: Trace-Based Distributed Deadlock Detection in Microservices

---

**Proposal under review:** *Trace-Based Distributed Deadlock and Livelock Detection in Asynchronous Microservice Architectures* — reconstructing temporal Wait-For Graphs (WFGs) from passive OpenTelemetry/Jaeger distributed traces, detecting persistent cyclic wait structures as the failure signature for distributed deadlock/livelock.

**Review method:** 2018–2026 literature sweep across five adversarial search angles (gap-invalidation, named-baseline verification, classical deadlock theory, "can traces evidence waiting" skepticism, practitioner/production evidence). 19 sources fetched, 92 falsifiable claims extracted, 25 top claims adversarially verified by independent 3-vote panels — **25 confirmed, 0 refuted**. Literature claims below are backed by verified citations; design sections (Parts 4–9) are synthesized on top of those verified constraints.

**Review date:** 2026-07-31

---

## Headline Finding (read this first)

---

Your gap is **partially supported — but one paper changes everything**. In October 2025, Rowicki, Francalanza & Scalas published "*Correct Black-Box Monitors for Distributed Deadlock Detection: Formalisation and Implementation*" (PACMPL/OOPSLA 2025, DOI [10.1145/3763069](https://doi.org/10.1145/3763069)), which explicitly targets RPC-based microservices, claims — verbatim — to be "*the first work that formalises, proves the correctness, and implements distributed black-box monitors for deadlock detection,*" proves its algorithm **sound and complete** with Coq-mechanised proofs, and ships an Erlang/OTP artifact (DDMon). This simultaneously *validates* your gap (nothing before 2025 did black-box microservice deadlock detection) and *occupies* the most quotable version of it.

The survivable position: their monitors are **active and co-deployed** — they observe live messages and exchange probes between monitors. They do not touch OpenTelemetry/Jaeger, recorded traces, offline analysis, sampling, livelock, or deadlock-vs-slowdown classification, and their guarantees hold only under a restricted single-threaded RPC (SRPC) model. Your proposal must be rebuilt around what they left open: **passive, sampled, uncertain telemetry** — and it must cite them prominently or a reviewer will do it for you.

---

## PART 1 — Gap Validation: Partially supported

---

**Verdict: Partially supported** (was "strongly supported" before OOPSLA 2025; is not "false/already solved" because the passive-telemetry formulation remains open).

All five verifiable named baselines were confirmed, by full-text or abstract+artifact greps, to contain **zero** occurrences of "deadlock", "livelock", "wait-for graph", or "cyclic wait", and none encodes blocked-waiting semantics:

| Paper             | Verified identity                                                                                                  | What it actually builds                                                                                                           |
|-------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Nezha</b>      | Yu et al., 2023, ESEC/FSE, DOI <a href="https://doi.org/10.1145/3611643.3616249">10.1145/3611643.3616249</a>       | Multi-modal <b>event graphs</b> for code-region/resource-type RCA                                                                 |
| <b>MicroRank</b>  | Yu et al., 2021, WWW, DOI <a href="https://doi.org/10.1145/3442381.3449905">10.1145/3442381.3449905</a>            | Spectrum-based latency root-cause <b>ranking</b> (personalized PageRank over trace coverage)                                      |
| <b>DeepTraLog</b> | Zhang et al., 2022, ICSE ( <a href="#">PDF</a> )                                                                   | Per-trace <b>event graph</b> (TEG: sequence / sync-request / sync-response / async-request edges) + GGNN anomaly scoring          |
| <b>Microscope</b> | Lin et al., 2018, ICSOC, DOI <a href="https://doi.org/10.1007/978-3-030-03596-9_1">10.1007/978-3-030-03596-9_1</a> | <b>Causality graph</b> from socket-syscall interception + SLO metrics — not OTel/Jaeger traces at all (it predates OpenTelemetry) |
| <b>Eadro</b>      | Lee et al., 2023, ICSE ( <a href="https://arxiv.org/abs/2302.05092">arXiv 2302.05092</a> )                         | Service <b>dependency/invoke graph</b> for joint anomaly detection + RCA                                                          |

**Sleuth, TraceGra, and CHASE could not be independently verified** in this review — treat your descriptions of them as unconfirmed until you locate the primary sources yourself. Notably, a 98-paper survey of microservice failure diagnosis ([arXiv 2407.01710](https://arxiv.org/abs/2407.01710), secondary source) mentions deadlock exactly once (as a cause inside "Resource Exhaustion"), never mentions "Wait-For Graph", "livelock", or "circular wait", and doesn't cite TraceGra or CHASE — strong absence-of-evidence support for the gap on the RCA side.

**The graph taxonomy you rely on is real and categorical** (verified against primary sources): Knapp 1987 (DOI [10.1145/45075.46163](https://doi.org/10.1145/45075.46163)) defines WFG edges verbatim as "*blocking relations between transaction agents*"; Kshemkalyani & Singhal ([Ch. 10](#)): an edge  $P1 \rightarrow P2$  exists iff "*P1 is blocked and is waiting for P2 to release some resource.*" Dependency graphs (Eadro), causal graphs (Microscope), and event graphs (Nezha/DeepTraLog) all encode "called / preceded / correlates-with" — never "is blocked on." No claim that RCA graphs are DAGs was asserted or needed.

**Two hard theoretical constraints your proposal currently violates:**

1. **Cycle  $\Leftrightarrow$  deadlock holds only in the AND/single-resource model** (Singhal 1989, DOI [10.1109/2.43525](https://doi.org/10.1109/2.43525)). In the OR/communication model — arguably closest to asynchronous, quorum-style microservice interactions — Knapp states verbatim: "*discovery of a cycle is insufficient for deadlock detection... a knot will indicate a deadlock [Holt 1972].*" A detector that flags cycles without fixing the request model is theoretically unsound.
  2. **The phantom-deadlock problem is your problem, amplified.** Knapp attributes phantom (false) deadlocks to "*message delays and out-of-date WFGs*" — and passive OTel telemetry is definitionally delayed (spans export after completion), sampled, and snapshot-inconsistent. Several published classical algorithms were later proven incorrect on exactly this basis (Gligor & Shattuck 1980, via Knapp). You inherit this problem in its worst form and must confront it explicitly.
-

## PART 2 — Ten Closest Prior Works, Ranked by Novelty Threat

| Rank | Paper                                                                                                | Year/Venue                                        | What It Does                                                                                                     | Similarity                                                                                  | Critical Difference                                                                                          | Threat      |
|------|------------------------------------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------|
| 1    | Rowicki, Francalanza & Scalas — <i>Correct Black-Box Monitors for Distributed Deadlock Detection</i> | 2025 OOPSLA, DOI 10.1145/3763069                  | Sound+complete black-box deadlock detection for RPC microservices; Coq-proven; Erlang artifact                   | Same problem (microservice deadlock), same black-box stance                                 | <b>Active</b> co-deployed probe-exchanging monitors, restricted SRPC model; no traces/OTel/sampling/livelock | <b>8/10</b> |
| 2    | Chandy-Misra-Haas + Knapp's four classes                                                             | 1983–87, ACM TOCS / CSUR, DOI 10.1145/45075.46163 | Probe-based edge-chasing; taxonomy of all active detection (path-pushing, edge-chasing, diffusing, global-state) | Same failure definition, same WFG formalism                                                 | All <b>active in-protocol</b> mechanisms assuming exact wait-for knowledge; none is post-hoc trace analysis  | 6/10        |
| 3    | CRISP — Zhang et al.                                                                                 | 2022 USENIX ATC ( <a href="#">link</a> )          | Critical-path analysis over Jaeger RPC traces at Uber; infers synchronization/wait on the critical path          | Closest proof that <b>passive spans can evidence waiting</b>                                | No cycles, no WFG, no deadlock/livelock claim anywhere                                                       | 5/10        |
| 4    | Nezha                                                                                                | 2023 FSE                                          | Multi-modal event-graph RCA                                                                                      | Trace-derived graphs, failure diagnosis                                                     | Event graph $\neq$ WFG; no blocking semantics                                                                | 4/10        |
| 5    | Eadro                                                                                                | 2023 ICSE                                         | Dependency-graph anomaly detection + RCA                                                                         | Trace-derived service graph                                                                 | Invocation edges, not wait edges; "deadlock" absent from full text                                           | 4/10        |
| 6    | DeepTraLog                                                                                           | 2022 ICSE                                         | TEG + GGNN anomaly detection                                                                                     | Encodes sync/async request structure per trace — nearly the raw material for wait inference | Learned anomaly scoring; no wait/cycle semantics                                                             | 4/10        |
| 7    | MicroRank                                                                                            | 2021 WWW                                          | Spectrum-based latency RCA                                                                                       | Trace-based, ranking faulty services                                                        | No graph-structural failure signature at all                                                                 | 3/10        |
| 8    | Metastable Failures in the Wild — Huang et al.                                                       | 2022 OSDI / <a href="#">;login:</a>               | 21 production metastable failures; >50% sustained by retry storms                                                | Covers the persistent-degradation space adjacent to livelock                                | Taxonomy/characterization, not detection; no WFG                                                             | 3/10        |

| Rank | Paper                                                                                              | Year/Venue                                                                                             | What It Does                                                                        | Similarity                                 | Critical Difference                                                     | Threat |
|------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------|--------|
| 9    | Genfer & Zdun — cyclic API dependencies                                                            | 2021 ECSA, DOI <a href="https://doi.org/10.1007/978-3-030-86044-8_15">10.1007/978-3-030-86044-8_15</a> | Detects cyclic microservice dependencies <b>statically</b> from source code         | Cycles in microservices                    | Static architecture analysis; no runtime waits                          | 2/10   |
| 10   | Astraea — <a href="#">BU/AWS</a> + adaptive per-span sampling ( <a href="#">arXiv 2107.07703</a> ) | 2024 / 2021                                                                                            | Probabilistic control of span sampling; shows traces are routinely partial per-span | Sampling-uncertainty machinery you'll need | Sampling control for latency diagnosis, not failure-structure inference | 2/10   |

**Has anyone already reconstructed Wait-For Graphs from distributed microservice traces for deadlock detection?**

**No** — with one adjacent **Partially**. No paper found in this review reconstructs WFGs from recorded OTel/Jaeger traces. The OOPSLA 2025 work detects microservice deadlocks black-box but via live co-deployed monitors, not trace reconstruction (verified against its abstract and [arXiv 2508.14851](#)). Caveat: this is an absence-of-evidence conclusion from a bounded sweep — 2024–2026 preprints, APM-vendor internal tooling, and non-English venues were not exhaustively certified.

## PART 3 — Hostile Reviewer Threats

1. **"OOPSLA 2025 already solved black-box microservice deadlock detection — with proofs you can't match."** → Strongest objection. → *Redesign*: position explicitly against it in §1: they require monitor deployment beside every service and a restricted SRPC model; you consume telemetry organizations already collect, at sampling rates they already run, and quantify the exactness you give up. Include their setting as a conceptual baseline.
2. **"A service-level cycle is not a deadlock."** → Verified theory agrees;  $A \rightarrow B \rightarrow A$  at service level is routine. → *Redesign*: vertices are request-scoped executions/resources (Part 6); edges require blocked-waiting evidence.
3. **"Spans don't evidence waiting — a long span may be computing, queueing, or GC-ing."** → This is your load-bearing assumption and the review found **no verified positive mechanism** for it. CRISP (ATC '22) is your best precedent that sync/wait structure is inferable from RPC spans. → *Redesign*: make wait-edge inference probabilistic, validated against ground truth from instrumented testbeds; report edge-inference precision/recall as a first-class result.
4. **"Cycles are the wrong condition under OR-model waits."** → Verified (Knapp/Holt). → *Redesign*: restrict soundness claims to synchronous-RPC AND-model waits; annotate edges with request-model semantics; detect knots for OR segments or scope them out explicitly.
5. **"Phantom deadlocks: your WFG is always stale."** → Verified as the classical safety failure. → *Redesign*: temporal persistence + confidence weighting as explicit phantom-suppression mechanisms, with measured false-deadlock rates.

6. **"Sampling destroys cycle evidence."** → One missing span breaks a cycle. → *Redesign*: make sampling-aware inference a headline contribution with degradation curves (Part 8).
  7. **"Livelock is unsupported."** → Verified production evidence (metastable-failures study: >50% of persistent failures sustained by retry storms, not lock cycles) shows the livelock-adjacent space is real but no passive livelock-detection precedent exists. → **Drop "livelock" from the title.** Keep retry-storm/circular-wait *discrimination* as a classification target instead.
  8. **"Deadlock by then is obvious from timeouts anyway."** → *Redesign*: measure time-to-detect and localization quality vs. timeout alerts; the value is identifying the *cycle core* (the OOPSLA paper itself motivates "blocked services obscuring the deadlock core").
  9. **"True distributed deadlocks are rare in production."** → DoorDash's failure taxonomy (cascading failure, retry storm, death spiral, metastable failure) never mentions deadlock. → *Redesign*: frame the detector as classifying the whole persistent-circular-wait family (including thread-pool-exhaustion stalls from synchronous call cycles, which practitioners *do* report), not only textbook deadlocks.
  10. **"Engineering integration, not research."** → *Redesign*: the research claims are the probabilistic wait-inference model, the sampling-robustness bounds, and the discrimination results — not the pipeline.
- 

## PARTS 4 & 5 — Enhanced Design and Algorithm

---

Keep your pipeline skeleton; the contribution moves into four mechanisms (your enhancement axes 1–5 and 7; axis 6, livelock, is dropped per Part 3, threat 7):

### Formal model.

- **Vertices**  $V = V_e \cup V_r$ : *execution vertices* (request-scoped span executions: trace-id, span-id, service instance) and *resource vertices* (logical resources inferable from instrumentation: connection pools, thread pools, locks, queue consumers).
- **Wait relation.**  $\text{wait}(u, v, [t_1, t_2])$ :  $u$  is blocked on  $v$  during the interval. Never observed directly; inferred.
- **Edge confidence.**  $c(e) = P(\text{wait} \mid \mathcal{E})$  from observable features: open client span with no progressing child, parent-child span overlap geometry, timeout/retry annotations, sync-vs-async span kind (OTel SpanKind + messaging semantics), pool-instrumentation events. Calibrated on ground-truth injected faults (this estimator is **novel**; the features are standard).
- **Model annotation.** Each edge carries  $m(e) \in \{\text{AND}, \text{OR}, \text{unknown}\}$ ; deadlock decision uses cycles over AND-only subgraphs and knots where OR edges participate (**novel as an applied mechanism**; theory is Holt/Knapp).
- **Temporal WFG.**  $G_t = (V, E_t)$ , maintained incrementally over sliding windows; incremental SCC maintenance is an *application* of existing dynamic-graph algorithms — claim no novelty there.
- **Cycle confidence & persistence.**  $C(\kappa) = \min_{e \in \kappa} c(e)$  (a chain of waits is only as credible as its weakest edge), and persistence  $P(\kappa, \Delta) = \text{fraction of windows in } [t_0, t_0 + \Delta] \text{ where } \kappa \text{ recurs with the same participant executions (same executions } \Rightarrow \text{ genuine stall; changing executions } \Rightarrow \text{ cyclic traffic)}$ .  
**Decision:** flag deadlock iff  $C(\kappa) \geq \theta_c \wedge P(\kappa, \Delta) \geq \theta_p$ , with  $(\theta_c, \theta_p, \Delta)$  swept in evaluation.

- **Sampling correction.** Under known head-sampling rate  $\rho$ , a cycle of length  $k$  survives intact in a per-trace-sampled store with probability  $\approx \rho$  (same-trace spans), but per-span adaptive sampling ([arXiv 2107.07703](#) confirms partial traces are the norm) requires estimating  $P(\text{cycle} \mid \text{observed partial edges}, \rho)$  — inflating edge posteriors for unobserved-but-implied waits. **This estimator is the second novel component.**

**Genuinely novel:** probabilistic wait-edge inference from passive spans; sampling-corrected cycle probability; request-model-annotated cycle/knot decision on inferred graphs; the deadlock-vs-slowdown-vs-retry-storm discrimination study.

**Not novel (say so in the paper):** Tarjan/Johnson/incremental SCC, OTel plumbing, temporal windowing.

---

## PART 6 — Graph Granularity

---

**Recommended: a two-layer graph — request-scoped execution vertices + logical-resource vertices, with service-level projection used only for explanation output.**

Service-level vertices are disqualified ( $A \rightarrow B \rightarrow A$  is routine and non-indicative; verified theory demands per-waiter blocking semantics — classical WFG nodes are *processes*). Pure span-level misses the resources (pools/locks) that actually mediate microservice circular waits — the practitioner-verified failure mode is synchronous call cycles exhausting thread pools. Threads are invisible to standard OTel; requests + instrumented resources are the finest granularity passive telemetry can support.

---

## PART 7 — Implementation Feasibility

---

**Yes — realistically implementable by a strong university student.** Software-only stack: Python (NetworkX + a small custom incremental-SCC layer), Docker Compose or k3s, OpenTelemetry SDKs, Jaeger, standard microservice benchmarks (e.g., DeathStarBench or Train Ticket), fault injection via toxiproxy/Chaos-Mesh-style delay and connection-pool starvation.

### Stages:

1. Testbed + deterministic deadlock injection harness — ~3–4 weeks
2. Trace ingestion + wait-edge feature extraction — ~3 weeks
3. Confidence model + temporal WFG + detection — ~4–6 weeks
4. Sampling experiments + baselines + ablation — ~4–6 weeks

**Main risks:** engineering a *genuine* reproducible distributed circular wait (hardest part — needs synchronous cyclic calls under bounded pools); ground-truth labeling; OTel async-messaging instrumentation gaps. Total  $\approx$  4–5 months full-time equivalent.

---

## PARTS 8 & 9 — Experiments and Ablation

---

**Scenarios** (each with ground-truth labels): genuine circular distributed wait (cyclic sync calls under bounded connection pools); ordinary slow RPC; slow DB call; transient cyclic interaction (async event loop, no blocking); timeout propagation chain; retry storm; partial trace loss; head sampling at 100/75/50/25/10%; async messaging if stage-2 instrumentation permits.

**Baselines:** latency-threshold alerting; timeout-based alerts; naive service-dependency-graph cycle detection (this baseline *must* lose on false positives — it's your thesis); naive WFG without temporal/confidence modeling; a trace-RCA baseline where implementable (MicroRank has a [public repo](#)).

**Metrics:** precision/recall/F1, FPR/FNR, time-to-detect vs. timeout alerts, localization accuracy of the cycle core, CPU/memory overhead, and the two robustness curves (detection quality vs. sampling rate; vs. span delay).

**Ablation:** A naive cycle detection → B +temporal persistence → C +confidence-weighted inference → D +sampling-aware correction → E full method. Each mechanism must show measurable, attributable improvement or be cut from the paper.

**The result that constitutes a contribution** (not just a prototype): *high precision (low phantom-deadlock rate) against the confusable scenarios — slow RPC, retry storms, transient cycles — sustained at  $\leq 25\%$  sampling, where every naive baseline collapses.* A detector that only works at 100% sampling on obvious deadlocks is engineering; the discrimination + degradation curves are the science.

---

## PART 10 — Final Enhanced Research Topic

---

### Titles (best → weakest)

1. *Probabilistic Wait-For Graph Reconstruction from Sampled Distributed Traces for Deadlock Detection in Microservice Systems*
2. *Detecting Persistent Circular Waits in Microservices from Passive, Incomplete Telemetry*
3. *Confidence-Weighted Temporal Wait-For Graphs: Trace-Based Distributed Deadlock Detection under Sampling*

### Research problem ( $\leq 150$ words)

Distributed deadlocks and deadlock-like stalls (synchronous call cycles exhausting connection pools) occur in microservice systems, where they manifest as cascades of blocked services that obscure the cycle core. Classical detectors (Chandy-Misra-Haas and all four of Knapp's algorithm classes) are active, in-protocol mechanisms assuming exact wait-for knowledge; the state-of-the-art black-box detector (OOPSLA 2025) requires monitors co-deployed beside every service. Production organizations, however, already collect passive OpenTelemetry traces — delayed, sampled, and incomplete. Whether persistent distributed circular waits can be detected, and discriminated from slow dependencies, timeout propagation, and retry storms,

using only this passive telemetry, is unknown. The problem is to reconstruct uncertainty-aware temporal Wait-For Graphs from sampled traces and decide deadlock with quantified confidence in place of the exactness passive data cannot provide.

## Verified research gap ( $\leq 200$ words)

Trace-based microservice diagnosis systems — Nezha (FSE '23), MicroRank (WWW '21), DeepTraLog (ICSE '22), Microscope (ICSOC '18), Eadro (ICSE '23) — build event, dependency, or causal graphs for RCA and anomaly detection; full-text verification finds no occurrence of deadlock, livelock, wait-for graphs, or cyclic-wait detection in any of them, and none encodes blocked-waiting semantics. A 98-paper failure-diagnosis survey mentions deadlock once, in passing. Classical theory (Knapp '87; Singhal '89) defines WFGs over blocking relations and detection via active protocols; it also establishes constraints any passive approach must answer: cycle-sufficiency holds only in the AND model, and stale graphs yield phantom deadlocks. Rowicki, Francalanza & Scalas (OOPSLA '25) provide the first correct black-box microservice deadlock detection — but via active, co-deployed, probe-exchanging monitors under a single-threaded-RPC model, with no treatment of recorded traces, sampling, incompleteness, or discrimination from performance pathologies. No prior work reconstructs Wait-For Graphs from passive distributed traces or characterizes deadlock detectability as a function of telemetry loss. That intersection — passive, sampled, confidence-weighted circular-wait detection — is unoccupied.

## Contributions

1. A probabilistic wait-edge inference model mapping OTel span evidence to calibrated  $P(\text{wait})$ .
2. Confidence-weighted temporal WFGs with request-model-annotated cycle/knot decision criteria and persistence-based phantom suppression.
3. A sampling-aware cycle-probability estimator with measured degradation bounds.
4. The first empirical study discriminating distributed circular waits from slow dependencies, timeout propagation, and retry storms in traces.
5. An open testbed + labeled fault corpus.

## Research questions

- **RQ1** — With what precision/recall can blocked-waiting relations be inferred from passive OTel spans?
- **RQ2** — Do temporal persistence and confidence weighting suppress phantom deadlocks relative to naive cycle detection?
- **RQ3** — How does detection quality degrade at 75/50/25/10% sampling, and can sampling-aware estimation recover it?
- **RQ4** — Can circular waits be discriminated from retry storms, timeout propagation, and slow dependencies using graph structure alone?

## Core hypothesis

Confidence-weighted, persistence-filtered temporal WFGs reconstructed from passive traces detect injected distributed circular waits with  $\geq 0.9$  precision and  $\geq 0.8$  recall at 100% sampling, retain  $\geq 0.7$  F1 at 25%



sampling with sampling-aware correction, and reduce false positives on confusable scenarios by  $\geq 50\%$  versus naive dependency-cycle detection.

## Methodology & evaluation

The Part 5 pipeline (span normalization → temporal ordering → probabilistic wait-edge inference → confidence-weighted temporal WFG → incremental cycle/SCC + knot detection → persistence scoring → sampling correction → deadlock probability → classification/explanation); validated by the Part 8 experiments and Part 9 ablation on an open benchmark testbed with injected, ground-truth-labeled faults.

---

## PART 11 — Publication Assessment (enhanced proposal)

---

| Category                              | Score /10 | Explanation                                                                                                                                                       |
|---------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Research Gap Strength                 | 7         | Real and verified, but narrowed by OOPSLA '25; gap is now "passive + sampled + discriminative," not "black-box microservice deadlock detection"                   |
| Novelty                               | 7         | Wait-inference-under-uncertainty and sampling-corrected detection are genuinely new; the failure definition and graph algorithms are classical                    |
| Technical Depth                       | 8         | Probabilistic inference + temporal graph theory + model-aware decision criteria is substantive                                                                    |
| Implementation Feasibility            | 8         | Fully software-only, standard stack, established benchmarks                                                                                                       |
| Experimental Evaluability             | 8         | Injected faults give clean ground truth; sampling sweeps are cheap and compelling                                                                                 |
| Practical Relevance                   | 7         | Uses telemetry orgs already have; tempered by practitioner evidence that pure deadlocks are rarer than retry storms                                               |
| Literature Differentiation            | 7         | Clean vs. RCA line; differentiation vs. OOPSLA '25 must be argued carefully, not assumed                                                                          |
| Reproducibility                       | 8         | Open-source stack + releasable fault corpus                                                                                                                       |
| Risk of Prior-Work Overlap (10 = low) | 6         | OOPSLA '25 is adjacent and its authors may extend toward traces; APM-vendor gray literature not exhaustively swept                                                |
| <b>Overall Publication Potential</b>  | <b>7</b>  | Solid second-tier venue target (e.g., ICSOC/Middleware/ICPE-class; SoCC/relevant tracks); top-tier possible only with exceptionally strong discrimination results |

**Implementation Difficulty: 6/10** — above a moderate university project because reproducing genuine distributed circular waits and calibrating the confidence model are hard; well below research-lab scale.

---

## PART 12 — Original vs. Enhanced Comparison

---

| Dimension              | Original Proposal | Enhanced Proposal |
|------------------------|-------------------|-------------------|
| Novelty                | 4/10              | 7/10              |
| Research Gap           | 5/10              | 7/10              |
| Technical Contribution | 4/10              | 7/10              |

| Dimension                 | Original Proposal | Enhanced Proposal |
|---------------------------|-------------------|-------------------|
| Implementation Difficulty | 4/10              | 6/10              |
| Publication Potential     | 4/10              | 7/10              |

**What raised the scores:** replacing deterministic call-cycle detection with **calibrated probabilistic wait inference**; making **sampling robustness** a first-class contribution with degradation curves; adding **request-model soundness** (AND-cycles vs. OR-knots) that no applied system has; converting the phantom-deadlock weakness into the **persistence/confidence mechanism**; **dropping livelock** (no telemetry support found; the metastable-failures literature shows livelock-like states are retry-sustained and better handled as a discrimination class); and honest positioning against OOPSLA '25.

**The enhanced proposal cannot legitimately reach 8–9/10 novelty.** The failure definition, WFG formalism, and detection algorithms are classical, and the "first black-box microservice deadlock detection" claim was taken in 2025. 7/10 is the honest ceiling.

---

## PART 13 — Publication Verdict: B — Proceed after specific modifications

---

**Final reviewer verdict:** The core insight — treating persistent cyclic waiting, rather than latency anomaly, as the failure signature recoverable from passive traces — is sound and occupies verified white space between classical active detectors, the OOPSLA 2025 active black-box monitor, and the trace-RCA literature. It is publishable *only* in the enhanced form: with probabilistic edge semantics, request-model-correct decision criteria, sampling-degradation results, and livelock removed. As originally framed ("build a WFG from traces, run cycle detection, detect deadlock and livelock"), it would be rejected on threats 1–4 and 7.

**Biggest novelty claim:** The first system to detect distributed circular-wait failures from passive, sampled distributed traces alone, with calibrated confidence in place of the co-deployed monitors and exactness assumptions all prior detectors require.

**Biggest weakness:** The load-bearing, currently unproven assumption that passive OTel spans contain enough signal to distinguish *blocked-waiting* from slow processing and queueing — no verified prior work establishes this in either direction (CRISP is the nearest supportive precedent). Validate it first; it is also the cheapest early kill-check.

**What must be demonstrated experimentally:** High-precision detection of injected circular waits that naive dependency-cycle detection cannot achieve — specifically, near-zero false positives on retry storms, timeout cascades, and transient cycles — *sustained at  $\leq 25\%$  trace sampling*.

**Kill criterion:** Discovery of any peer-reviewed or credible industrial work that reconstructs wait/blocking graphs from recorded distributed traces for deadlock or hang detection — the most likely sources being a follow-up by the OOPSLA 2025 authors extending DDMon to offline traces, or an APM-vendor system (Datadog/Dynatrace/Meta/Google-internal) surfacing in gray literature. Re-run that specific search before committing, and again before submission.

---

## Verified Citation List

| Paper                                                                                           | Authors                       | Year | Venue                 | DOI/URL                                                                                              | Status                                     |
|-------------------------------------------------------------------------------------------------|-------------------------------|------|-----------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------|
| Correct Black-Box Monitors for Distributed Deadlock Detection: Formalisation and Implementation | Rowicki, Francalanza & Scalas | 2025 | PACMPL/OOPSLA         | <a href="https://doi.org/10.1145/3763069">10.1145/3763069</a> ; <a href="#">arXiv 2508.14851</a>     | Verified (ACM record via arXiv mirror)     |
| Deadlock Detection in Distributed Systems (survey)                                              | E. Knapp                      | 1987 | ACM Computing Surveys | <a href="https://doi.org/10.1145/45075.46163">10.1145/45075.46163</a>                                | Verified, full text                        |
| Deadlock Detection in Distributed Systems                                                       | M. Singhal                    | 1989 | IEEE Computer         | <a href="https://doi.org/10.1109/2.43525">10.1109/2.43525</a>                                        | Verified via mirrors                       |
| Distributed Computing: Principles, Algorithms, and Systems — Ch. 10                             | Kshemkalyani & Singhal        | 2008 | Cambridge Univ. Press | <a href="#">Chapter PDF</a>                                                                          | Verified, full text                        |
| Nezha: Interpretable Fine-Grained Root Cause Analysis for Microservices                         | Yu et al.                     | 2023 | ESEC/FSE              | <a href="https://doi.org/10.1145/3611643.3616249">10.1145/3611643.3616249</a> ; <a href="#">repo</a> | Verified (abstract + repo)                 |
| MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis                | Yu et al.                     | 2021 | WWW                   | <a href="https://doi.org/10.1145/3442381.3449905">10.1145/3442381.3449905</a> ; <a href="#">repo</a> | Verified (abstract + repo)                 |
| DeepTraLog: Trace-Log Combined Microservice Anomaly Detection                                   | Zhang et al.                  | 2022 | ICSE                  | <a href="#">PDF</a>                                                                                  | Verified, full text                        |
| Microscope: Pinpoint Performance Issues with Causal Graphs in Microservice Environments         | Lin et al.                    | 2018 | ICSOC                 | <a href="https://doi.org/10.1007/978-3-030-03596-9_1">10.1007/978-3-030-03596-9_1</a>                | Verified (abstract + survey corroboration) |
| Eadro: An End-to-End Troubleshooting Framework for Microservices                                | Lee et al.                    | 2023 | ICSE                  | <a href="https://arxiv.org/abs/2302.05092">arXiv 2302.05092</a>                                      | Verified, full text                        |
| CRISP: Critical Path Analysis of Large-Scale Microservice Architectures                         | Zhang et al. (Uber/UCSB)      | 2022 | USENIX ATC            | <a href="#">link</a>                                                                                 | Verified                                   |
| Metastable Failures in the Wild                                                                 | Huang et al.                  | 2022 | OSDI / ;login:        | <a href="#">link</a>                                                                                 | Verified                                   |
| Detecting Cyclic Dependencies between Microservices (API-level, static)                         | Genfer & Zdun                 | 2021 | ECSA (Springer)       | <a href="https://doi.org/10.1007/978-3-030-86044-8_15">10.1007/978-3-030-86044-8_15</a>              | Verified                                   |

| Paper                                                    | Authors           | Year | Venue            | DOI/URL                    | Status                                                                   |
|----------------------------------------------------------|-------------------|------|------------------|----------------------------|--------------------------------------------------------------------------|
| Astraea: online probabilistic tracing / sampling control | BU PEAC Lab + AWS | 2024 | —                | <a href="#">PDF</a>        | Verified                                                                 |
| Adaptive per-span sampling (partial-trace prevalence)    | —                 | 2021 | arXiv (preprint) | <a href="#">2107.07703</a> | Preprint — label as such                                                 |
| Microservice failure-diagnosis survey (98 papers)        | —                 | 2024 | arXiv (preprint) | <a href="#">2407.01710</a> | Secondary/preprint — label as such                                       |
| <b>Sleuth, TraceGra, CHASE</b>                           | —                 | —    | —                | —                          | <b>NOT independently verified — locate primary sources before citing</b> |

---

## Review Caveats

---

- The OOPSLA 2025 paper was verified via its arXiv mirror (2508.14851) against the paywalled ACM record; published Oct 2025 — follow-on work extending it toward passive traces could appear at any time and would collapse the remaining gap.
- Nezha and Microscope were verified by abstract + official repositories + survey corroboration, not word-for-word full text.
- Sleuth, TraceGra, and CHASE remain unverified.
- "No prior WFG-from-traces work found" is an absence-of-evidence conclusion from a bounded sweep (2024–2026 preprints, industrial/APM gray literature, and non-English venues not exhaustively certified), not a proof.
- All 25 claims that survived verification did so on adversarial 3-vote panels with none refuted; one claim survived 2–1.
- Parts 4–9 (algorithm, granularity, feasibility, experiments, ablation) are design synthesis on top of the verified constraints, not independently fact-checked literature.